

NLP-Assignment 1

Yonatan Karni | Raz Ziv

May 26, 2026

3.

a.

Firstly we notice that for fixed o, c the number of times $p_\theta(o|c)$ shows in the objective is $\#(c, o)$,

This follows from the fact that in each window where both o, c is in we count $p_\theta(o|c)$ one more time.

$$\mathcal{L}(p_\theta(c|o)) = \#(c, o) \log(p_\theta(o|c))$$

Then we can rewrite the objective as:

$$\mathcal{L}(\theta) = \sum_t \sum_{-m \leq j \leq m} \log(p_\theta(o|c)) = \sum_{c \in V_{oc}} \sum_{o \in V_{oc}} \#(c, o) \log(p_\theta(o|c)) = \sum_{c \in V_{oc}} \sum_{o \in V_{oc}} \mathcal{L}(p_\theta(c|o))$$

For a fixed o, c we must make sure $\sum_{o' \in V_{oc}} p_\theta(o'|c) = 1$ namely θ is a probability distribution.

Therefore we will use Lagrange multiplier with λ parameter to enforce this constraint.

Moreover, we can see that maximizing the original objective $\mathcal{L}$ is equivalent to maximize each term in the sum over the words o, c .

$$\begin{aligned} \theta^* &= \arg \max_{\theta} \mathcal{L}(\theta) \\ \iff \forall (c, o) \quad p_{\theta^*}(o|c) &= \arg \max_{p_\theta(c|o)} \mathcal{L}(p_\theta(c|o), \lambda) \\ s.t \quad \mathcal{L}(p_\theta(c|o), \lambda) &= (\#(c, o)) \log(p_\theta(o|c)) + \lambda \left(1 - \sum_{o' \in V_{oc}} p_\theta(o'|c) \right) \end{aligned}$$

Differentiating by $p_\theta(o|c)$ and comparing to 0 we get:

$$\begin{aligned} \frac{\partial \mathcal{L}(p_\theta(c|o), \lambda)}{\partial p_\theta(o|c)} &= \frac{\#(c, o)}{p_{\theta^*}(o|c)} - \lambda = 0 \\ p_{\theta^*}(o|c) &= \frac{\#(c, o)}{\lambda} \end{aligned}$$

Now substitute back into the original Lagrange multiplier definition:

$$\begin{aligned} \sum_{o' \in V_{oc}} p_{\theta^*}(o'|c) &= \sum_{o' \in V_{oc}} \frac{\#(c, o')}{\lambda} = 1 \\ \lambda &= \sum_{o' \in V_{oc}} \#(c, o') \end{aligned}$$

Finally we obtain:

$$p_{\theta^*}(o|c) = \frac{\#(c, o)}{\sum_{o' \in V_{oc}} \#(c, o')}$$

b.

We will take the following corpus $\{abc, cbc, aac\}$ over the alphabet $\{a, b, c\}$

Using a window of size 3 for skip-gram model.

Let us assume p is optimal while being expressed with the given softmax expression.

Lets calculate the optimal probabilities using the optimal form we found in last section:

$$\begin{aligned} p(a|c) &= \frac{3}{8}, \quad p(a|a) = \frac{2}{6} \\ p(b|c) &= \frac{3}{8}, \quad p(b|a) = \frac{1}{6} \\ p(c|c) &= \frac{2}{8}, \quad p(c|a) = \frac{3}{6} \end{aligned}$$

Now we can obtain the following on the embedding of v_c, v_a :

$$\begin{aligned} \log \left(\frac{p(a|c)}{p(c|c)} \right) &= \log \left(\frac{e^{u_a v_c}}{e^{u_c v_c}} \right) = v_c (u_a - u_c) = \log \left(\frac{3}{2} \right) \\ \log \left(\frac{p(a|a)}{p(c|a)} \right) &= \log \left(\frac{e^{u_a v_a}}{e^{u_c v_a}} \right) = v_a (u_a - u_c) = \log \left(\frac{2}{3} \right) \\ \iff \frac{v_c}{v_a} &= \frac{\log \left(\frac{3}{2} \right)}{\log \left(\frac{2}{3} \right)} \end{aligned}$$

On the other hand:

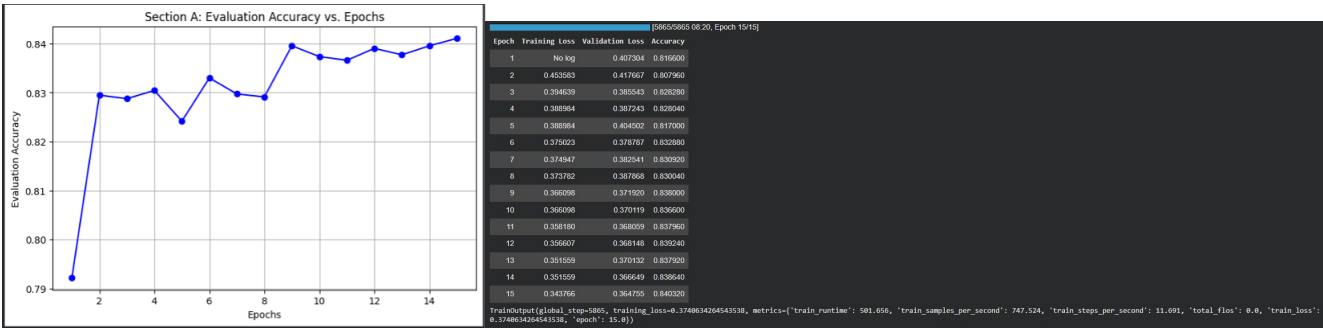
$$\begin{aligned} \log \left(\frac{p(b|c)}{p(c|c)} \right) &= \log \left(\frac{e^{u_b v_c}}{e^{u_c v_c}} \right) = v_c (u_b - u_c) = \log \left(\frac{3}{2} \right) \\ \log \left(\frac{p(b|a)}{p(c|a)} \right) &= \log \left(\frac{e^{u_b v_a}}{e^{u_c v_a}} \right) = v_a (u_b - u_c) = \log \left(\frac{1}{3} \right) \\ \iff \frac{v_c}{v_a} &= \frac{\log \left(\frac{3}{2} \right)}{\log \left(\frac{1}{3} \right)} \end{aligned}$$

From both we get the following contradiction:

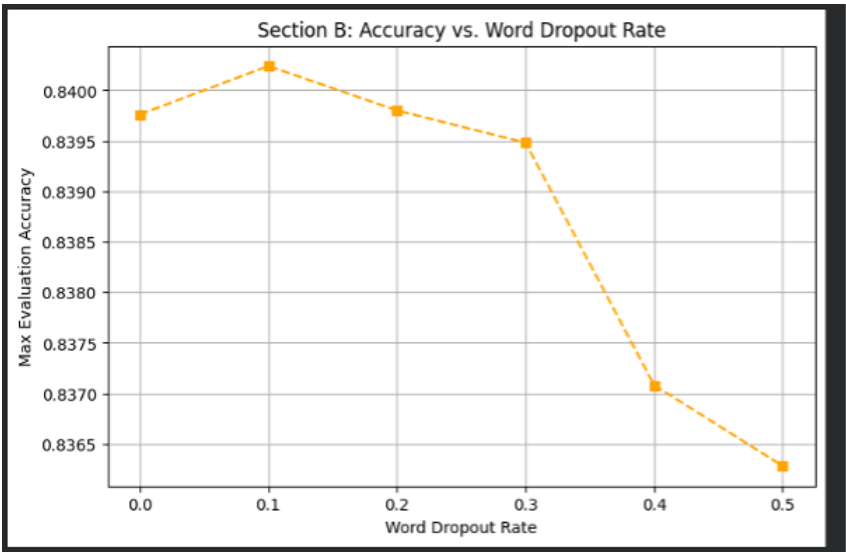
$$\frac{\log \left(\frac{3}{2} \right)}{\log \left(\frac{2}{3} \right)} = \frac{\log \left(\frac{3}{2} \right)}{\log \left(\frac{1}{3} \right)} \iff \log(3) = \log(2)$$

4.

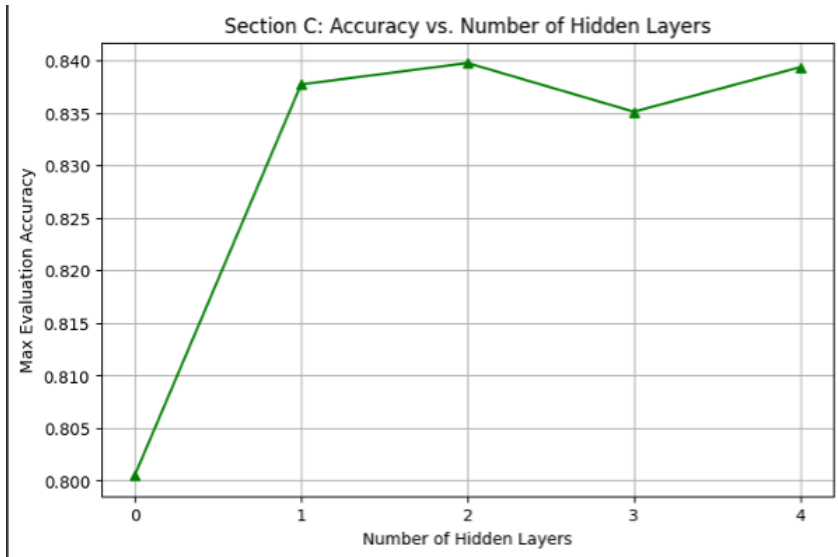
a.



b.



c.

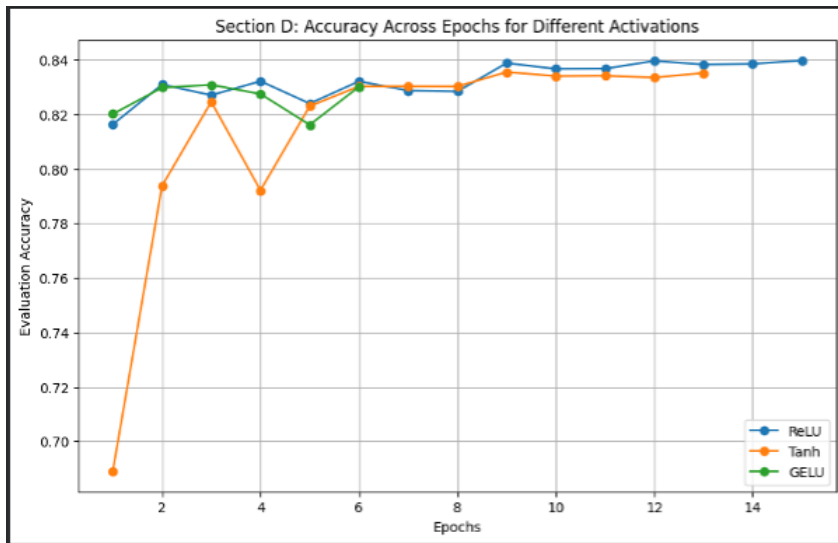


Based on the graph from Part C, the effect of diminishing returns can be observed immediately after the first and second hidden layers. Adding one layer significantly boosts the accuracy from around 0.800 to nearly 0.838. The accuracy reaches its peak at 2 hidden layers (approaching 0.840), drops slightly at 3 layers, and rises again a little at 4 layers. These results are completely expected.

A DAN model works by computing a simple average of the word embeddings, meaning that the input to the network is a simple, orderless representation of the text. While adding one or two layers helps the network learn non-linear combinations of these averaged features, adding too many layers (such as 3 or 4) to such a simple and orderless input often leads to overfitting or instability, which explains why the performance stabilizes or decreases slightly.

The linear model (0 hidden layers) did not achieve better performance than the 4-hidden-layer model. The graph clearly shows that the model with 0 hidden layers had the lowest accuracy in the group (approximately 0.800), while the 4-hidden-layer model reached an accuracy of about 0.839.

d.



The choice of activation function significantly affects the initial learning dynamics and the convergence speed of the network. ReLU and GELU demonstrated very similar performance: both immediately jumped to high accuracy (above 0.81) already in the first epoch, while maintaining stability throughout the entire training process.

Tanh, on the other hand, struggled at the beginning. It started with a much lower accuracy of approximately 0.689 and experienced a sharp drop around epoch 4, before eventually catching up with the other functions in the later stages of training.

5.

5.1.a.

I will discuss two intrinsic evaluation metrics:

Coherence-

measure how much a certain topic is coherent.

Namely, how much top words that belong to the topic are likely to be seen near each other in the corpus.

For example, given the following words inside a topic:

“surf sea wave shore” will give higher coherence than “surf champion banana car”

simply because the first words are more likely to appear in the same context.

The mathematical expression for the coherence:

First we will choose some top matching words W for a certain topic T .

Denote $P(w)$ the probability to get the word w from some context, and $P(u, w)$ is the joint distribution, ie to see both words in the same context.

$$\text{Coherence}(T) = \frac{1}{|W|} \sum_{(w,u) \in W} \log \left(\frac{P(w, u)}{P(w) P(u)} \right)$$
$$\text{Coherence} = \frac{1}{K} \sum_{i=1}^K \text{Coherence}(T_i)$$

Diversity-

The diversity metric aims to measure how much the topics differ.

We measure how likely that a word will appear in both topics in the following method:

Choose W_k set of most likely words for each topic.

Assume K topics and N top words per topic.

$$\text{Diversity} = \frac{\left| \bigcup_{k=1}^K W_k \right|}{K \cdot N}$$

5.1.b.

I saw several topic models use wikipedia as a benchmark dataset for evaluation.

20 Newsgroup, a large dataset contains news posts on 20 topics is used in different research papers.

Trump’s Twitter messages archive was used in BERTopic evaluation git repository.

5.3.

I used the two coherence parameters c_v and c_uci .

Both parameters use a sliding window to check if some words appear together, c_v is more sophisticated with additional statistical methods and different normalization.

I got the following results:

$$c_v = 0.41$$
$$c_uci = -5.97$$

The first parameter range $0 \leq c_v \leq 1$ where 0 means no coherence at all and 1 means very strong coherence. We got 0.41 so we can see a coherence was detected but not that high, probably due to real world messy text data or not enough data.

The second parameter $c_uci \in \mathbb{R}$ however most of times it falls approximately in $[-15, 5]$.

It shows a medium coherence measure which probably results from its strict window of length 10, some words appear for example in the next sentence.

5.4

One problem is semantics, namely the literal meaning of words. The two evaluation parameters I have shown earlier can fail on this regard.

For example a topic may contain top words which are written close to each other many times and therefore produce high coherence, but the words are very different semantically.

Another example is a model with very low diversity which may seem bad. But a human may recognize a lot of topics being merged together in the corpus, for example science of music.